

Code Explanation

AURA Payload Parsing and Transformation Code Explanation

The provided Python code is responsible for parsing and transforming AURA payloads from base64 to canonical XML. It takes in a base64-encoded AURA payload, a policy number, and paths to XSLT files for Aura 14 and Aura 15, and returns the parsed AURA output along with the policy number.

Overview of the Code

The code implements a multi-step process to decode and transform the input AURA payload. It first checks if the input is null or empty, then performs base64 decoding and JSON preprocessing. The code then enters a progressive decoding loop to decode the payload until it becomes valid XML. After that, it detects whether the XML is in canonical form and selects the appropriate XSLT file for transformation. Finally, it normalizes the XML, applies the XSLT transformation, and returns the result.

Step 1: Input Validation

The code starts by checking if the input `i_aura_input_base64` is null or empty. If so, it logs a warning and returns `None` along with the `policy_number`.

```
if not i_aura_input_base64 or i_aura_input_base64.strip() == '':
    logging.warning("AURA input is null or empty")
    return None, policy_number
```

Step 2: Base64/JSON Preprocessing

The code attempts to base64 decode the input payload and checks if the resulting output is JSON. If it is, the code extracts the `interviewDetails` from the JSON data if present.

```
try:
    aura_output = base64.b64decode(i_aura_input_base64).decode('utf-8')
    if aura_output.startswith('{'):
        try:
            json_data = json.loads(aura_output)
            if 'interviewDetails' in json_data:
                aura_output = json_data['interviewDetails']
                logging.info("Extracted interviewDetails from JSON")
        except json.JSONDecodeError as e:
            logging.error(f"Failed to parse JSON: {str(e)}")
except Exception as e:
    logging.error(f"Initial base64 decode failed: {str(e)}")
    return None, policy_number
```

Step 3: Progressive Decoding Loop

The code enters a loop where it continuously base64 decodes the `aura_output` until it becomes valid XML or reaches a maximum of 100 iterations.

```
decode_count = 0
while not aura_output.startswith('<') and decode_count < 100:
    try:
```

```

        decoded = base64.b64decode(aura_output).decode('utf-8')
        aura_output = decoded
        decode_count += 1
        logging.debug(f"Decoded layer {decode_count}")

        if aura_output.startswith('<'):
            break
    except Exception as e:
        logging.error(f"Failed at decode iteration {decode_count}: {str(e)}")
        break

```

Step 4: Canonical XML Detection

After the decoding loop, the code checks if the `aura\_output` is valid XML. If not, it logs a warning and returns `None` along with the `policy\_number`.

```

if not aura_output.startswith('<'):
    logging.warning("AURA input was not valid XML after decoding")
    return None, policy_number

```

Step 5: XSLT Selection Rule

The code checks if the `aura\_output` contains the string "TXLife" to determine whether to use the Aura 15 XSLT or the Aura 14 XSLT.

```

if "TXLife" in aura_output:
    xslt_path = aura15_xslt_file_path
    logging.info("Using Aura 15 XSLT")
else:
    xslt_path = xslt_file_path
    logging.info("Using Aura 14 XSLT")

```

Step 6: XML Normalization

The code removes the XML declaration encoding attribute and wraps the `aura\_output` in an `` container.

```

aura_output = re.sub(r'(<?.*)encoding(.*?>)', r'12', aura_output)
wrapped_xml = f"<XMLA>{aura_output}</XMLA>"

```

Step 7: XSLT Transformation

The code loads the selected XSLT file, parses the `wrapped\_xml`, applies the XSLT transformation, and returns the result along with the `policy\_number`.

```

try:
    xslt_root = etree.parse(xslt_path)
    transform = etree.XSLT(xslt_root)
    xml_root = etree.parse(StringIO(wrapped_xml))
    result_tree = transform(xml_root)

```

```
    parsed_aura_output = str(result_tree)
    logging.info("After AURA transformer for Conversion")
    return parsed_aura_output, policy_number
except Exception as e:
    logging.error(f"XSLT transformation failed: {str(e)}")
    return None, policy_number
```